

Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

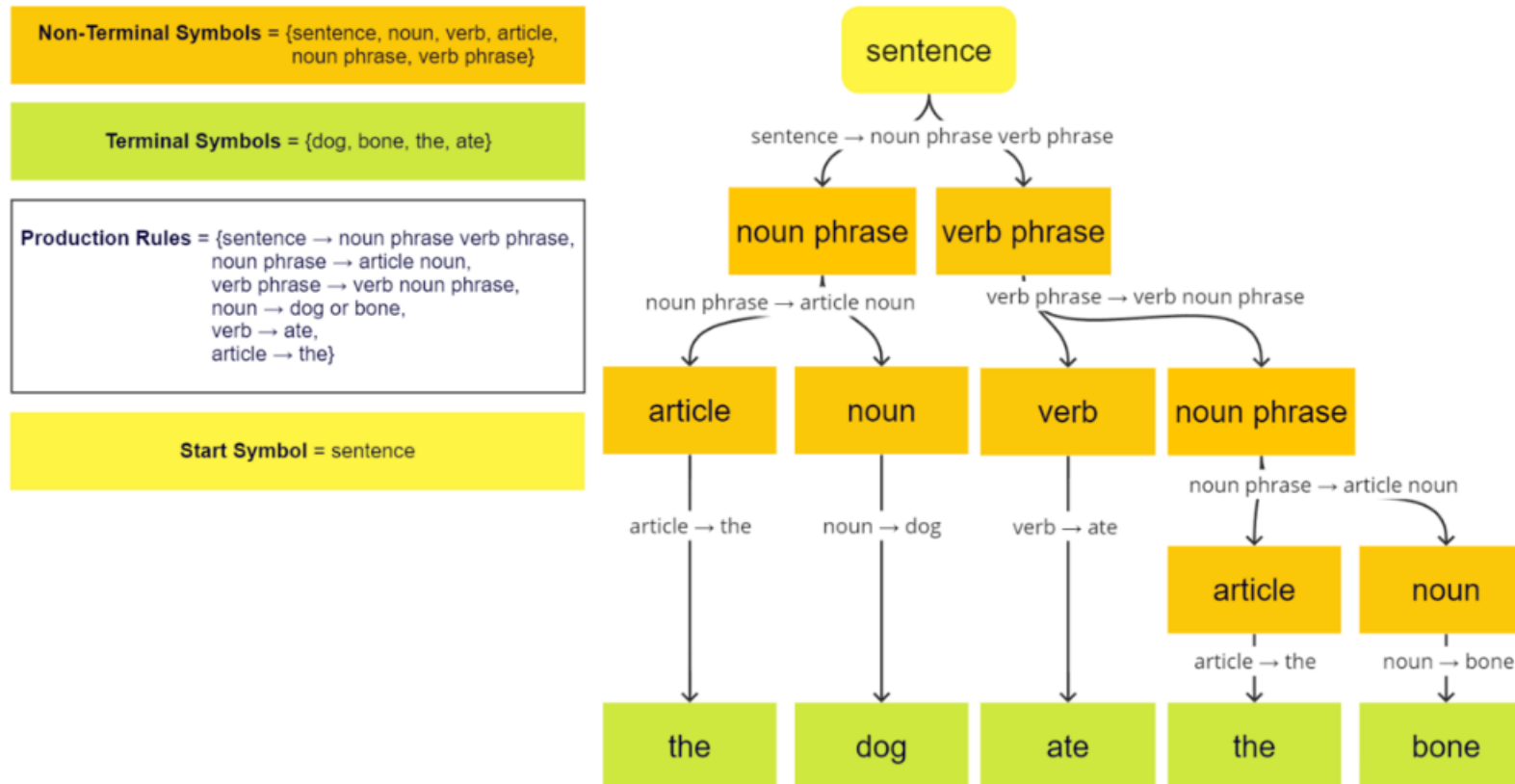
Email: minhaz.ahmed@gmail.com

Content

- Context Free Grammar
- Parse tree
- Ambiguity
- Parser
- Lexical Analyzer
- Top down
- Bottom-up parser
- Operator precedence parsing

Context Free Grammar (CFG)

- ▶ Context free grammar is a formal grammar which is used to generate all possible strings in a given formal language.



Advantages of CFG

- ▶ There are the various usefulness of CFG:
 - Context free grammar is useful to describe most of the programming languages.
 - If the grammar is properly designed then an efficient parser can be constructed automatically.
 - Using the features of associativity & precedence information, suitable grammars for expressions can be constructed.
 - Context free grammar is capable of describing nested structures like: balanced parentheses, matching begin-end, corresponding if-then-else's and so on.

Context Free Grammar (CFG)

- ▶ A CFG consists of a finite set of grammar rules is a quadruple (N, T, P, S)
- ▶ N is a set of non terminal symbol
- ▶ T is a set of terminal symbol
- ▶ P is a set of rules
- ▶ S is the start symbol

$S \rightarrow Aa$, $A \rightarrow Ab|c$
CFG

Example:

- ▶ $N = \{S, A\}$
- ▶ $T = \{a, b, c\}$
- ▶ $P = \{S \rightarrow Aa, A \rightarrow Ab, A \rightarrow c\}$
- ▶ $S = S$

Context Free Grammar (CFG)

► Production rules:

$S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow c$

check that **abbcbbba** string can be derived from the given CFG.

$S \Rightarrow aSa$

$S \Rightarrow abSba$

$S \Rightarrow abbSbba$

$S \Rightarrow abbcbbba$

Context Free Grammar (CFG)

Example 5.5

Consider the grammar with productions

$$\begin{aligned} S &\rightarrow aAB, \\ A &\rightarrow bBb, \\ B &\rightarrow A|\lambda. \end{aligned}$$

Then

$$S \Rightarrow aAB \Rightarrow abBbB \Rightarrow abAbB \Rightarrow abbBbbB \Rightarrow abbbbB \Rightarrow abbbb$$

is a leftmost derivation of the string *abbbb*. A rightmost derivation of the same string is

$$S \Rightarrow aAB \Rightarrow aA \Rightarrow abBb \Rightarrow abAb \Rightarrow abbBbb \Rightarrow abbbb.$$

Categories of CFG

► Non Recursive CFG

$S \rightarrow Aa$ output: {ba, ca}

$S \rightarrow Aa$
 $\rightarrow ba$

$A \rightarrow b|c$

► Recursive CFG

$S \rightarrow Aa$ output: { ca, cba, cbba, cbbba, }

$S \rightarrow Aa$
 $\rightarrow Aba$
 $\rightarrow cba$

$A \rightarrow Ab|c$

Context Free Grammar (CFG)

Example of a CFG

$S \rightarrow Aa$

$A \rightarrow Ab \mid c$

Input: {c, b, b, a}

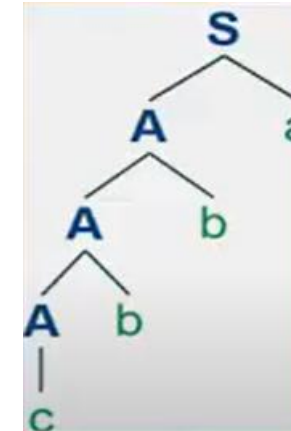
$S \rightarrow Aa$

$\rightarrow Aba$

$\rightarrow Abba$

$\rightarrow cbba$

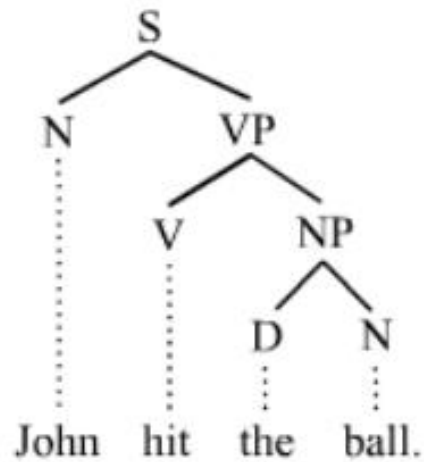
Parsing using CFG



Parse tree

Parse tree

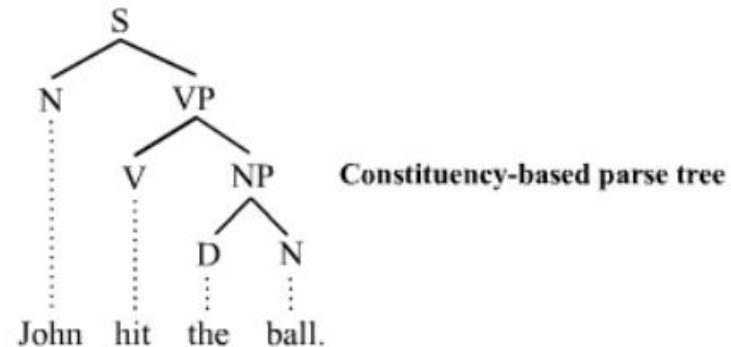
- ▶ A parse tree or parsing tree or derivation tree or concrete syntax tree is an ordered, rooted tree that represents the syntactic structure of a string according to some context-free grammar. The term parse tree itself is used primarily in computational linguistics;



Constituency-based parse tree

Parse tree

- ▶ The constituency-based parse trees of constituency grammars (phrase structure grammars) distinguish between terminal and non-terminal nodes. The interior nodes are labeled by non-terminal categories of the grammar, while the leaf nodes are labeled by terminal categories.



S for **sentence**, the top-level structure in this example
NP for **noun phrase**. The first (leftmost) NP, a single noun "John", serves as the **subject** of the sentence.
The second one is the **object** of the sentence.
VP for **verb phrase**, which serves as the **predicate**
V for **verb**. In this case, it's a **transitive verb** *hit*.
D for **determiner**, in this instance the **definite article** "the" N for **noun**

Derivation

- ▶ Derivation is a sequence of production rules. It is used to get the input string through these production rules. During parsing we have to take two decisions. These are as follows:
 - We have to decide the non-terminal which is to be replaced.
 - We have to decide the production rule by which the non-terminal will be replaced.

We have two options to decide which non-terminal to be replaced with production rule.

Left-most Derivation

- ▶ In the left most derivation, the input is scanned and replaced with the production rule from left to right. So in left most derivatives we read the input string from left to right.

Production

$S = S + S$

$S = S - S$

$S = a \mid b \mid c$

Input:

a - b + c

Left-most Derivation

► The left-most derivation is:

$$S = S + S$$

$$S = S - S + S$$

$$S = a - S + S$$

$$S = a - b + S$$

$$S = a - b + c$$

Right-most Derivation

- ▶ In the right most derivation, the input is scanned and replaced with the production rule from right to left. So in right most derivatives we read the input string from right to left.

$$S = S + S$$

$$S = S - S$$

$$S = a \mid b \mid c$$

- ▶ Input: $a - b + c$

The right-most derivation is:

$$S = S - S$$

$$S = S - S + S$$

$$S = S - S + c$$

$$S = S - b + c$$

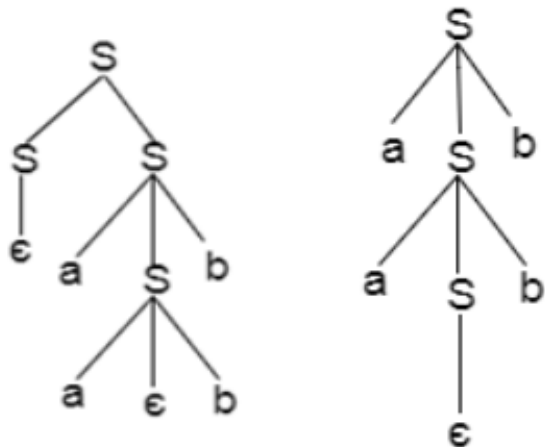
$$S = a - b + c$$

Ambiguity

A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivative or more than one parse tree for the given input string. If the grammar is not ambiguous then it is called unambiguous.

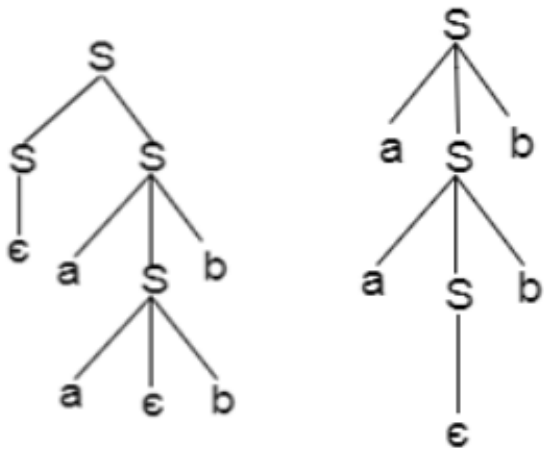
$$S = aSb \mid SS$$
$$S = \epsilon$$

For the string aabb, the above grammar generates two parse trees:



Ambiguity

- ▶ If the grammar has ambiguity then it is not good for a compiler/IDE construction. No method can automatically detect and remove the ambiguity but you can remove ambiguity by re-writing the whole grammar without ambiguity.



Ambiguity example

Example grammar:

$S \rightarrow S + S$

$S \rightarrow S * S$

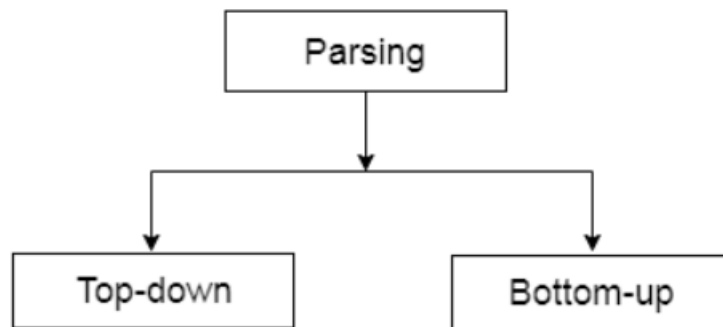
$S \rightarrow \text{NUMBER}$

Input String: $1 + 2 * 3$

$$\begin{array}{c} + \\ / \backslash \\ 1 \quad * \\ / \backslash \\ 2 \quad 3 \end{array}$$
$$\begin{array}{c} * \\ / \backslash \\ + \quad 3 \\ / \backslash \\ 1 \quad 2 \end{array}$$

Parser

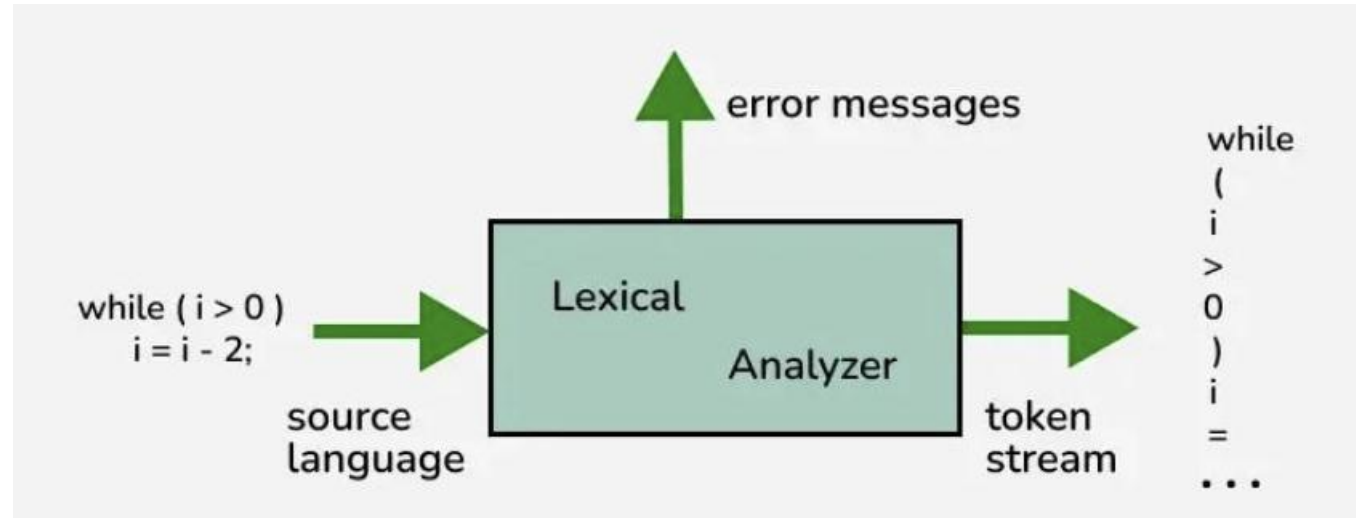
- ▶ Parser is a compiler that is used to break the data into smaller elements coming from lexical analysis phase.
- ▶ A parser takes input in the form of sequence of tokens and produces output in the form of parse tree.
- ▶ Parsing is of two types: **top down** parsing and **bottom up** parsing.



Parser

- ▶ Parsing analyzing a sequence of tokens to determine the grammatical structure of a program
- ▶ **Role of Parser:**
- ▶ Context-free syntax analysis: The parser checks if the structure of the code follows the basic rules of the programming language (like grammar rules). It looks at how words and symbols are arranged.
- ▶ Guides context-sensitive analysis: It helps with deeper checks that depend on the meaning of the code, like making sure variables are used correctly.
- ▶ Constructs an intermediate representation: The parser creates a simpler version of your code that's easier for the computer to understand and work with.
- ▶ Produces meaningful error messages: If there's something wrong in your code, the parser tries to explain the problem clearly so you can fix it.

Lexical analyzer



Lexical analysis, also known as scanning is the first phase of a compiler which involves reading the source program character by character from left to right and organizing them into tokens. Tokens are meaningful sequences of characters.

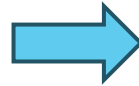
A scanner, or lexical analyzer, uses a Deterministic Finite Automaton (DFA) to recognize these tokens, as DFAs are designed to identify regular languages.

Lexical analyzer

- ▶ Suppose we pass a statement through lexical analyzer: **a = b + c;**
- ▶ It will generate token sequence like this: **id=id+id;**

```
int main()  
{  
  // 2 variables  
  int a, b;  
  a = 10;  
  return 0;  
}
```

C program



```
'int' 'main' '(' ')' '{' 'int' 'a' ';' 'b' ';' 'a' '=' '10' ';' 'return' '0' ';' '}'
```

Equivalent token

Advantages of Lexical analysis

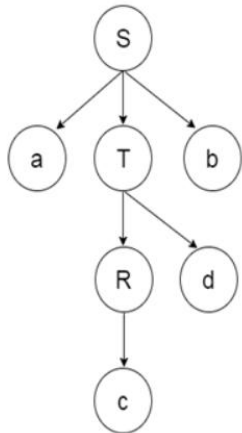
- **Simplifies Parsing:** Breaking down the source code into tokens makes it easier for computers to understand and work with the code. This helps programs like compilers or interpreters to figure out what the code is supposed to do. It's like breaking down a big puzzle into smaller pieces, which makes it easier to put together and solve.
- **Error Detection:** Lexical analysis will detect lexical errors such as misspelled keywords or undefined symbols early in the compilation process. This helps in improving the overall efficiency of the compiler or interpreter by identifying errors sooner rather than later.
- **Efficiency:** Once the source code is converted into tokens, subsequent phases of compilation or interpretation can operate more efficiently. Parsing and semantic analysis become faster and more streamlined when working with tokenized input.

Disadvantages of Lexical analysis

- **Limited Context:** Lexical analysis operates based on individual tokens and does not consider the overall context of the code. This can sometimes lead to ambiguity or misinterpretation of the code's intended meaning especially in languages with complex syntax or semantics.
- **Overhead:** Although lexical analysis is necessary for the compilation or interpretation process, it adds an extra layer of overhead. Tokenizing the source code requires additional computational resources which can impact the overall performance of the compiler or interpreter.
- **Debugging Challenges:** Lexical errors detected during the analysis phase may not always provide clear indications of their origins in the original source code. Debugging such errors can be challenging especially if they result from subtle mistakes in the lexical analysis process.

Top-down parsing

- The top down parsing is known as recursive parsing or predictive parsing.
- Bottom up parsing is used to construct a parse tree for an input string.
- In the top down parsing, the parsing starts from the start symbol and transform it into the input symbol.
- ▶ Parse Tree representation of input string "acdb" is as follows:



Bottom-up parsing

- ▶ Bottom-up parsing is also known as shift-reduce parsing.
- ▶ Bottom-up parsing is used to construct a parse tree for an input string.
- ▶ In the bottom-up parsing, the parsing starts with the input symbol and construct the parse tree up to the start symbol by tracing out the rightmost derivations of string in reverse.

$E \rightarrow T$

$E \rightarrow T * F$

$T \rightarrow id$

$F \rightarrow T$

$F \rightarrow id$

Bottom-up parsing

- Parse Tree representation of input string "id * id" is as follows:

id * id

Step 1

T * id
|
F
|
id

Step 4

F * id
|
id

Step 2

T
/ | \
T * F
| |
F id
|
id

Step 5

T * id
|
F
|
id

Step 3

E
|
T
/ | \
T * F
| |
F id
|
id

Step 6

Bottom-up parsing

Bottom-up parsing is classified in to various parsing. These are as follows:

1. Shift-Reduce Parsing
2. Operator Precedence Parsing
3. Table Driven LR Parsing

Shift reduce parsing

- Shift reduce parsing is a process of reducing a string to the start symbol of a grammar.
- Shift reduce parsing uses a stack to hold the grammar and an input tape to hold the string.
- At the shift action, the current symbol in the input string is pushed to a stack.
- At each reduction, the symbols will be replaced by the non-terminals. The symbol is the right side of the production and non-terminal is the left side of the production.

Example

► Grammar

$S \rightarrow S+S$

$S \rightarrow S-S$

$S \rightarrow (S)$

$S \rightarrow a$

Input string: $a1-$
 $(a2+a3)$

Parsing table

Stack contents	Input string	Actions
\$	a1-(a2+a3)\$	shift a1
\$a1	-(a2+a3)\$	reduce by $S \rightarrow a$
\$S	-(a2+a3)\$	shift -
\$S-	(a2+a3)\$	shift (
\$S-(	a2+a3)\$	shift a2
\$S-(a2	+a3)\$	reduce by $S \rightarrow a$
\$S-(S	+a3) \$	shift +
\$S-(S+	a3) \$	shift a3
\$S-(S+a3	) \$	reduce by $S \rightarrow a$
\$S-(S+S	) \$	shift)
\$S-(S+S)	\$	reduce by $S \rightarrow S+S$
\$S-(S)	\$	reduce by $S \rightarrow (S)$
\$S-S	\$	reduce by $S \rightarrow S-S$
\$S	\$	Accept

Operator precedence parsing

- ▶ Operator precedence grammar is kinds of shift reduce parsing method. It is applied to a small class of operator grammars.

There are the three operator precedence relations:

- $a \succ b$ means that terminal "a" has the higher precedence than terminal "b".
- $a \preceq b$ means that terminal "a" has the lower precedence than terminal "b".
- $a \doteq b$ means that the terminal "a" and "b" both have same precedence.

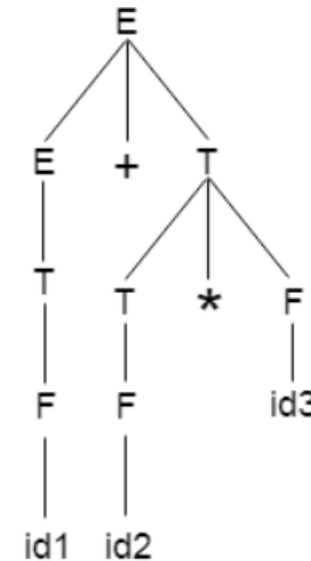
Operator precedence parsing

Given string:

$w = \text{id} + \text{id} * \text{id}$

On the basis of above tree, we can design following operator precedence table:

	E	T	F	id	+	*	\$
E	X	X	X	X	$\doteq$	X	$\triangleright$
T	X	X	X	X	$\triangleright$	$\doteq$	$\triangleright$
F	X	X	X	X	$\triangleright$	$\triangleright$	$\triangleright$
id	X	X	X	X	$\triangleright$	$\triangleright$	$\triangleright$
+	X	$\doteq$	$\triangleleft$	$\triangleleft$	X	X	X
*	X	X	$\doteq$	$\triangleleft$	X	X	X
\$	$\triangleleft$	$\triangleleft$	$\triangleleft$	$\triangleleft$	X	X	X



Equivalent parse tree

Operator precedence parsing

$\$ \prec \text{id1} \succ + \text{id2} * \text{id3} \$$

$\$ \prec F \succ + \text{id2} * \text{id3} \$$

$\$ \prec T \succ + \text{id2} * \text{id3} \$$

$\$ \prec E \doteq + \prec \text{id2} \succ * \text{id3} \$$

$\$ \prec E \doteq + \prec F \succ * \text{id3} \$$

$\$ \prec E \doteq + \prec T \doteq * \prec \text{id3} \succ \$$

$\$ \prec E \doteq + \prec T \doteq * \doteq F \succ \$$

$\$ \prec E \doteq + \doteq T \succ \$$

$\$ \prec E \doteq + \doteq T \succ \$$

$\$ \prec E \succ \$$

Accept.

	E	T	F	id	+	*	\$
E	X	X	X	X	$\doteq$	X	$\succ$
T	X	X	X	X	$\succ$	$\doteq$	$\succ$
F	X	X	X	X	$\succ$	$\succ$	$\succ$
id	X	X	X	X	$\succ$	$\succ$	$\succ$
+	X	$\doteq$	$\prec$	$\prec$	X	X	X
*	X	X	$\doteq$	$\prec$	X	X	X
\$	$\prec$	$\prec$	$\prec$	$\prec$	X	X	X

let us process the string with the help of the above

Constituency Parsing

- ▶ Constituency parsing is a natural language processing technique that is used to analyze the grammatical structure of sentences. It is a type of syntactic parsing that aims to identify the constituents, or subparts, of a sentence and the relationships between them. The output of a constituency parser is typically a parse tree, which represents the hierarchical structure of the sentence.
- ▶ The process of constituency parsing involves identifying the syntactic structure of a sentence by analyzing its words and phrases. This typically involves identifying the noun phrases, verb phrases, and other constituents, and then determining the relationships between them. The parser uses a set of grammatical rules and a grammar model to analyze the sentence and construct a parse tree.

Constituency Parsing

- ▶ Constituency parsing aims to identify the hierarchical structure of a sentence by grouping words into *constituents* or *phrases*. These constituents represent grammatical units like noun phrases (NPs), verb phrases (VPs), prepositional phrases (PPs), etc



S (Sentence): The topmost node represents the entire sentence.

NP (Noun Phrase): "The cat" and "the mat" are noun phrases. They consist of a determiner (Det) and a noun (N).

VP (Verb Phrase): "sat on the mat" is a verb phrase. It consists of a verb (V) and a prepositional phrase (PP).

PP (Prepositional Phrase): "on the mat" is a prepositional phrase. It consists of a preposition (P) "on" and a noun phrase (NP) "the mat".

Det (Determiner): "The" and "the" are determiners (articles).

N (Noun): "cat" and "mat" are nouns.

V (Verb): "sat" is the verb.

Dependency Parsing

- ▶ Dependency parsing focuses on the *relationships* between individual words in a sentence. It identifies which words depend on which other words, forming a **directed graph of dependencies**. The output of a dependency parser is typically a dependency tree or a graph, which represents the relationships between the words in the sentence.
- ▶ The process of dependency parsing involves identifying the syntactic relationships between words in a sentence. This typically involves identifying the subject, object, and other grammatical elements, and then determining the relationships between them. The parser uses a set of grammatical rules and a grammar model to analyze the sentence and construct a dependency tree or graph.

Dependency Parsing



ROOT: "sat" is the root of the sentence—the main verb. All other words depend on it in some way.

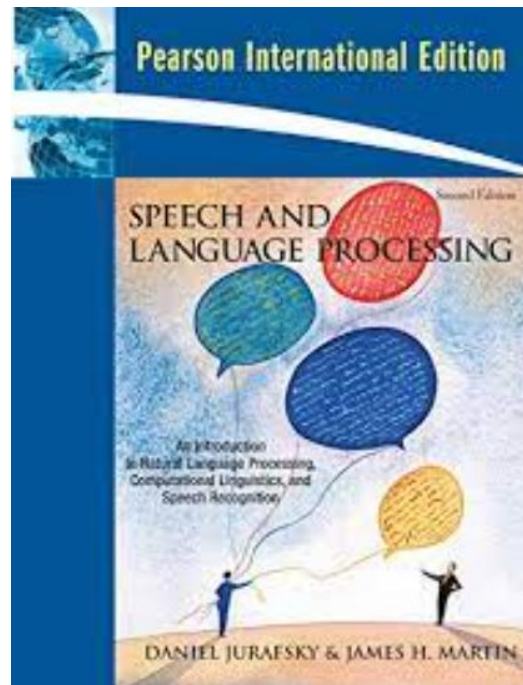
nsubj (nominal subject): "cat" is the nominal subject of the verb "sat." It's the one doing the sitting.

det (determiner): "The" (both instances) are determiners, modifying the nouns "cat" and "mat."

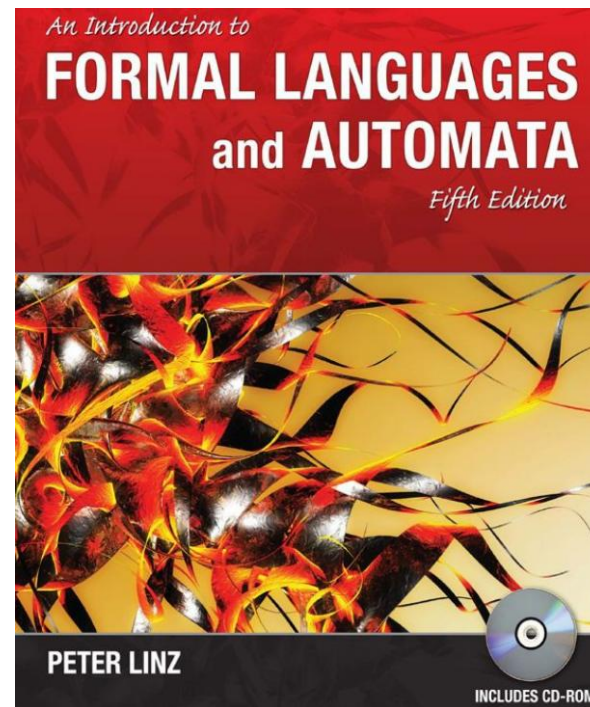
prep (preposition): "on" is a preposition.

pobj (object of preposition): "mat" is the object of the preposition "on." It's what the cat is sitting on.

Reference



Chapter 2



Chapter 5

Question



Thank you

